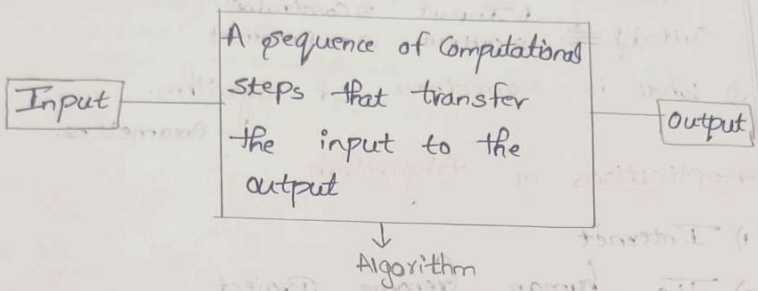
Dt:- 15/7/26

UNIT:01 :- Introduction to Algorithm

Algorithm



Def :- (2M)

An Algorithm is a sequence of instructions for solving the Problem, it is the step-by-step procedure, with the input to solve the Problem in Finite no. of time to obtain the required Output.

Characteristics of Algorithm :- (2M)

1) Finiteness → According to output
2) Definiteness → Related to Input
3) Input (non-zero input)
4) Output (non-zero output)
5) Effectiveness (correctness of particular Output)

(*) The Role of Algorithm in Computing

1) What is instance of a Problem & Input + constraint (output ఉండాలి exact condition means)

2) How to decide which Algorithm is best Suited? ⇒ 1) Input 2) output 3) Constraint 4) environment (Possible env to run) (best algorithm ని ఎలా ఎంచుకోవాలి)

3) What is Correctness of algorithm. (parameters.)

Applications of Algorithm

1) Internet
2) The human Genome Project (about DNA matching)
3) e-Commerce
4) Page rank
5) Weather Forecasting
6) Linear Programming → (used for reservation systems)
7) Shortest Path Algorithm.

Dt: 16/07/26

(*) Steps in Problem Solving | in writing = Algorithm

1) Analysing the Problem → Input Problem & constraint output
2) Designing Algorithm
3) Analysis of Algorithm
4) Implementation (or) Coding
5) Testing & Debugging.

Designing Algorithms (Methods) :-

1) Brute force Method

→ It is a straightforward approach to a problem which iteratively identifies every possibility to solve the problem.

Example :-

2) Recursive Algorithm

→ Based on Recursion.

→ A problem is solved by breaking into sub-problems of the same type & calling again till problem get solved.

Example :- (1) Factorial of No.

(2) Tree Traversal

(3) DFS

3) Divide & Conquer technique

→ Divide the Problem Conquer them, & merge - again

1) Divide :- Divide to subproblem

2) Conquer :- Solve them recursively

3) Combine :- Combine solution at last

Example :- (1) Quick Sort

(2) Merge Sort.

4) Greedy Approach
→ It is Used to Solve the Optimization Problem.
→ choosing the best Solution on each iteration.
→ It doesn't guarantees optimal Solution.
→ Prism's, kruskal's, Dijikstra's.

5) Dynamic Programming
→ Remembering the result of Previous run & using them to arrive at new results
→ knapsack Problem, Job Scheduling Problem.

6) Backtracking Algorithm
→ Finding a Solution in incremental manner.
→ If Unsuccessful move at any point then it will backtrack again to root Node.

Example :- 1) N-Queen
2) Graph Colding.

Module - II 17/07/26

Analysis of Algorithms

* Complexities of Algorithm

1) Time Complexity
2) Space Complexity.

Space Complexity :-

- The amount of Temporary storage required for Running the algorithm
- The Amount of memory needed to perform the Operation
- "The total amount of Computer Memory required by an Algorithm to complete its execution. is called Space Complexity"
- Memory Utilization for 3 reasons
  1) Instruction space
  2) Environmental stack
  3) Data space → variables, Constants & Control structures.

Example :-

```
int Sum = 0;
for (int i=0; i<n; i++)
{
  Sum = Sum + arr[i]
}
```

Sum ⟶ 4 bytes
int ⟶ 4 bytes
4+4 = 8 bytes
S(n) = 8 (bytes)
S(n) = O(1) → Space complexity.

ఇక్కడ మనకి int, & value 4 bytes చొప్పున రెండూ కాబట్టి O(1). అని అర్థం

ఎన్ని calculate చెయ్యాలో కొద్దీ మనకి దొన్నీ O(n) అని అన్నాం

Simply :-
O(1) → చెప్పే వాళ్లలోనే
O(n) → చెప్పేట్టుగా
↓
అంటే 'n' number అని అర్థం.

Time Complexity :- 2M | 7M | 8M

- The Amount of time need to run the Program is termed as 'time efficiency' or Time Complexity
- Total time Considered on the basis of Compile time & Run time.
- "The time Complexity of an algorithm is the total amount of time required to Complete the execution.
- Time Efficiency based on :
  1) Operation Counts
  2) Step Counts
  3) Asymptotic Notations.

Example :-

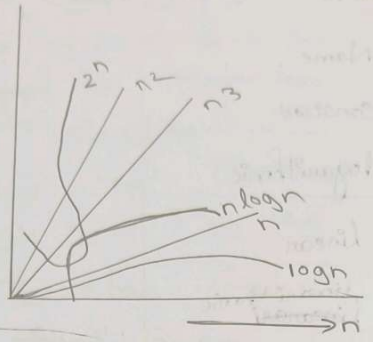
step count → A = a*b; ← operation count	This take 1 Unit of Time O(1)
for (i=0; i<n; i++) a = a+i; → operation count	This take n unit of time O(n)
for (i=0; i<n; i++) for (j=0; j<n; j++) Printf("Hello"	This take n^2 unit of time. O(n^2)

Order of Growth

Function	Name
1	Constant
$\log(n)$	logarithmic
n	linear
$n \log n$	linearithmic
n^2	quadratic
n^3	cubic
2^n	exponential
$n!$	Factorial

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

n	logn	nlogn	n^2	n^3	2n
1	0	0	1	1	2
2	1	2	4	8	4
4	2	8	16	64	16
8	3	24	64	512	256



Dt: 22/07/26

Best Case, Worst Case, Average Case

Let us find the Complexity function $T(n)$ for Certain Cases.

1) Worst Case : It gives maximum Value of $T(n)$ for any possible input

2) Best Case : It gives the Minimum Value of $T(n)$ for any possible input.

3) Average Case : It gives expected value of $T(n)$.

Def's

1) Worst Case Efficiency :

The worst case efficiency of an Algorithm is for worst case input size n, where Algorithm runs longest among all possible inputs of that size

2) Best-case Efficiency

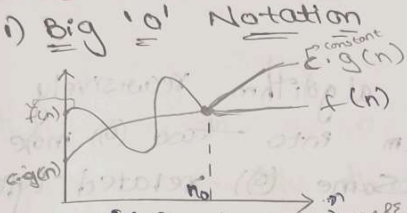
The Best case Efficiency of an Algorithm is for best case input size of n, for which algorithm run's fastest among all possible inputs of that size.

3) Avg case Efficiency :-

The Avg case is neither a wost case nor the best case, it is situated in between lower and upper bound of an Algorithm.

Asymptotic Notations

1) Big 'O' Notation



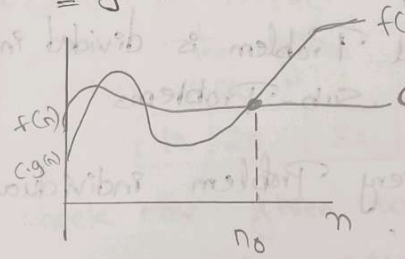
f(n) యొక్క position పైన ఉంటే best case లేకపోతే wost case

$f(n) = O(g(n))$ → బిగ్ O g(n) ఉంటే చూడాలి

upper bound → wost case

$f(n) \leq c \cdot g(n)$

2) Big Ω Notation బిగ్ O Notation పైన దానికి ఇది (వ్యతిరేకం)

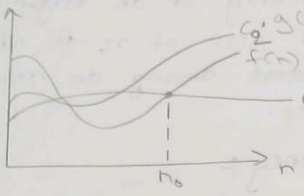


$f(n) = (\Omega\ g(n))$

lower bound → best case

$f(n) \geq c \cdot g(n)$

3) Big 'Θ' Notation



$f(n) = \Theta(g(n))$

$$c_1 g(n) \leq f(n) \leq c_2 \cdot g(n)$$

(Theory Part Completed)

Divide & Conquer

→ A divide & conquer algorithm recursively breaks down a problem into two (or) more sub problems of the same (or) related type, until these become simple enough to solved directly.

It consists of steps

1) Divide :- The Original Problem is divided into a set of Sub-Problems

2) Conquer :- Solve every Problem individual recursively.

Combine :- Put together solutions of the Sub Problems to get the solution.

Fundamentals of Divide & Conquer Strategy

1) Relational formula
2) Stopping Condition.

Applications / methods of divide & Conquer

1) Binary search
2) Quick sort
3) Merge sort.

Dt: 23/07/26

Binary Search :-

Binary search is a searching algorithm that operates on a sorted (or) monotonic search space, repeatedly dividing it into the two parts to find a target value (or) optimal answer in a logarithmic time.

Example :- 7M/8M

Solve the given example using Binary search key = 40.

10	20	30	40	50	60

1) check the given array is sorted (or) not, if array is sorted apply the divide & Conquer method.

2) Assign the index positions to array & apply divide & Conquer method by finding mid element

$mid = \frac{low + high}{2}$

3)

0	1	2	3	4	5
10	20	30	40	50	60

$Mid = 0+5/2$

$= 2.5 \approx 2$

Hence the new mid is '30' & index-Value is '2'

4) Check new mid is equal to element (8) not

$30 \neq 40$

$30 < 40$

Hence do the search operation in right side array, hence do mid+1, again find the new-mid.

The New-mid is $mid = L + H/2$

$= 3+5/2$

$= 8/2 = 4$

The New mid is '50' with index Value '4'

→ Compare with key-Value $40 > 50$, Hence again-Search is left side of an array & find new-mid.

5) New mid $= low + high/2$

$= \frac{3+3}{2} = 6/2 = 3$

Hence the new mid is Value of '40' with index Position '3' i.e $40 = 40$

c) As the Searching element matches the key-value hence as per binary search of given array
→ Element '40' found at location '3'

Example 2 :- | 2 | 3 | 7 | 8 | 11 | 15 | 25 |

Binary Search Algorithm (7M | 8M)

```
Binary Search (A, n, key)
  low ← 0
  high ← n-1
  while low ≤ high do
    mid ← (low + high)/2
  then if A[mid] = key
    return mid
  else if A[mid] < key
  then
    low ← mid + 1
  else
    high ← mid - 1
  end while
  return -1
```

Time Complexity :-

Best Case :- O(1)

Worst Case :- O(log n)

Space Complexity :-

O(log n)

Dt:- 24/01/26

Merge Sort :-

The Merge Sort Algorithm is a divide & Conquer algorithm that sorts an Array by first breaking it down into Smaller arrays. & building the array back together the correct way so that it is sorted.

How it Works :-

1) Divide the Unsorted array into two sub-arrays, half the size
2) Continue to divide the Sub-arrays as long as the Current Piece of the array
3) Merge two Sub-arrays togather by always putting the lowest value first
4) keep Merging until there are no Sub-arrays left.

Example :-

12	8	9	3	11	5	4

1) Divide the array into Sub-parts using divide & Conquer method & identify the mid.

$$mid = \frac{low + high}{2}$$

12	8	9

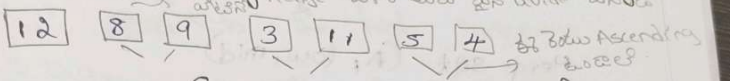
3	11	5	4

Hence we will get two - Sub-arrays.

2) check the array & again divide the array to the make subparts.

12	8 | 9	3 | 11	5 | 4

3) We need to divide the array till we do-not get the single element in each array



4) Now start Conquering & combining the array using backward step same as step No-3 & also arrange them in a ascending order while combining

12	8 | 9	3 | 11	4 | 5

5) Now again Merge the array to make less number of subparts as step 1

8 | 9 | 12	3 | 4 | 5 | 11

6) Now again Merge the array till we didn't get the final solution & arrange them in a ascending order.

3	4	5	8	9	11	12

Hence final sorted array is

3	4	5	8	9	11	12

Example:-

26	27	43	3	9	82	10

(Tree లా రాయాలి రఫ్ బుక్ లో ఉంది).

Algorithm/ pseudocode for Merge sort

Merge Sort (A, low, high)
if low < high then → సరిగా Array element ఉందా లేదా అని చెక్ చెయ్యటానికి
mid ← (low+high)/2
Merge sort (A, low, mid) → మనం రెండు భాగాలు చేసాం, రెండో భాగం కోసం
Merge sort (A, mid+1, high) → ఇది రెండో భాగం కోసం
Merge sort (A, low, mid, high) → రెండింటిని కలపటానికి

Time Complexity → n times రివైడ్ చెస్తూంటా ఉంటుంది.
Best, Average, worst case → O(n log n)

Space Complexity
O(n)

Dt: 29/07/26

Quick Sort 10M

- Quick Sort is a Sorting Algorithm
- It sorts the given data item in ascending order.
- It uses the idea of divide & Conquer.
- It follows the recursive Algorithm.

How to Apply Conditions in quick sort ?

1) choose the highest index Value as a 'Pivot'
   ↳ any element.
2) Take two Variables to Point left & right to the list excluding Pivot.
3) Left Points to the low index
4) Right Points to the high index
5) while Value at left is less than 'Pivot' move right
6) While Value at right is greater than 'Pivot' move left
7) If both step 5 & 6 doesn't match Swap left & right.

What is Pivot Element ?

It is element from an array made for Comparison. The choice of pivot can be (first element, median, random element, (&) last element).

Example :-

19	17	15	12	16	18	4	11	13

Partition 1	key	Partition 2

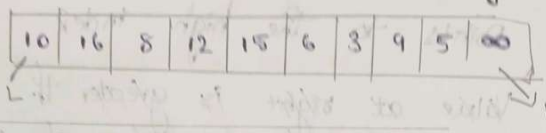
Values < key Pivot Values > key

Solve the given example using Quick Sort 8|16M
Part-C

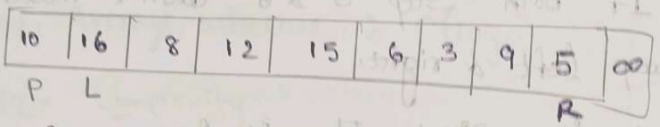
10	16	8	12	15	6	3	9	5

Solution Pass 1

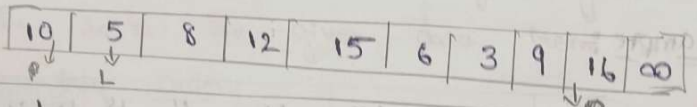
1) Lets Consider 10 as pivot element. Also 10 as left element & '∞' as right element.



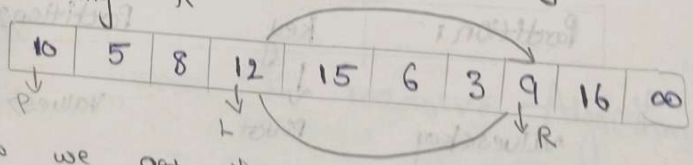
2) Start incrementing L till we didn't get higher element than pivot element & start decrementing 'R' till we didn't get the element - lower than pivot element.



Stop incrementing L Such as (L) 16 is greater than Pivot & start incrementing right, the right (5) is less than pivot.
- Swap 'L' & 'R' with each other. Swap 16 & 5



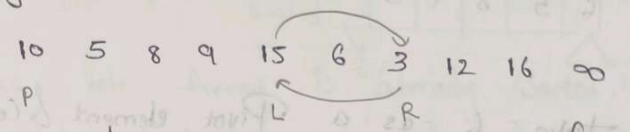
3) Now again start incrementing L & decrementing R



- Now we get the element 12 & 9 for L&R respectively. Swap 12 & 9

10	5	8	9	15	6	3	12	16	∞
P			L				R		

4) Now again Start incrementing L & decrementing R.



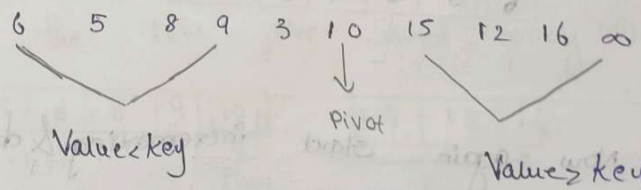
Swap the L & R as we get the stopping conditions.

10	5	8	9	3	6	15	12	16	∞
P				L		R			

5) Repeat the ~~Product~~ Procedure for increment & decrement L & R.

10	5	8	9	3	6	15	12	16	∞
P					R	L			

Now whenever L & R cross each other Swap 'Right element' with 'Pivot' element.



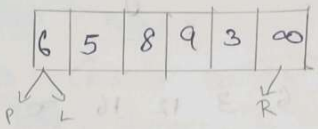
Pass 2 = [R, L cross అయితే, R ని Pivot తో swap చెయ్యాలి]

As we get ~~the~~ ~~proper~~ proper place for the Pivot element, The Smaller & greater elements are situated right & left of the mid.

Now Sort each of the array Separately by

applying recursive quick-sort Method.

Left-side-Array :-



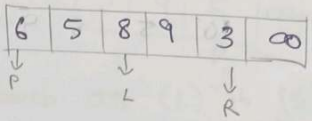
Now take 6 as a Pivot element & Compare 6 with low & high elements.

Low = 6 High = ∞

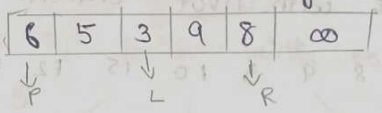
Increment Low till we get greater element than 6 & decrement Right till we get the lesser element than 6.

L will stop at 8

R will stop at 3



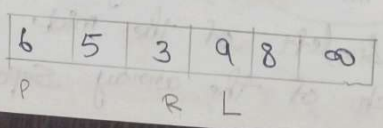
→ Swap 8 & 3 The array will be



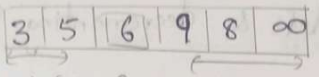
2) Now again start incrementing & decrementing L & R

L = 3 & R = 8

L will stop at 9 & R will start decrement & stop at 3

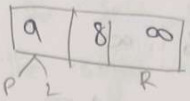


As element R Cross the left element swap Pivot & Right swap 6 & 3

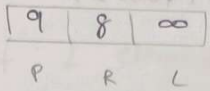


3) As left Array is already Sorted. Sort the right Subarray

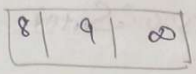
→ Now take 9 as Pivot & left & ∞ as Right element



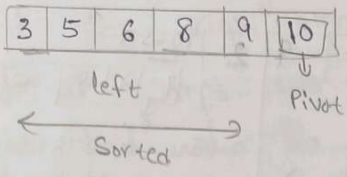
L = ∞
R = 8

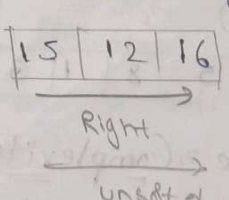


As Right element Cross left element Swap ~~Patter~~ Position of Pivot & Right, Hence array will be



Hence the left side Sorted array will be





Pass - 3

1) Now the right Sub-array is Remaining to sort. Hence assign the left, right & Pivot to right Sub array.

15	12	16	∞

P L R

Start incrementing left & decrementing R, according to pivot

15	12	16	∞

P R L

As right Cross the left element Swap Pivot & right Swap 15 & 12 Now array will be Sorted.

12	15	16	∞

Now including Pass 1, Pass 2 & Pass 3 the sorted Array is

3	5	6	8	9	10	12	15	16

left pivot right

∴ Hence the array is Sorted using Quick Sort.

Ex (2)

5	8	1	3	7	9	2

Pass 3

7	9	8	∞

P

Compare 9 with pivot 7, 9 > 7

Compare 8 with pivot 7, 8 > 7

"No element is smaller than '7', so the pivot is already in correct position"

(2) Sort right sub array [9, 8]

Take 9 as pivot

9	8	∞

P R L

Swap right with pivot

[8 | 9]

Time Complexity

Best Case $O(n \log n)$

Worst Case $O(n^2)$

Average $O(n \log n)$

Space Complexity

Average / Best case ⟶ $O(n \log n)$

Worst Case ⟶ $O(n)$.

Algorithm

1) Start
2) Choose a Pivot
3) Select a left pointer & right-Pointer
4) Compare the left element (L element) with the Pivot & the right element (R element) with the Pivot
5) Check if L element < pivot &
   R element > pivot
   → If yes, increment the left Pointer & decrement right Pointers
   → If not, Swap the 'L' element & 'R' - element.
6) When left ≥ right, Swap the pivot element with the right element (pointer)
7) Repeat steps 1 to 5 on the left half & right half of the list till entire list is sorted
8) Stop.